

Unit 5

Templates:

Generic programming is an approach where generic types are used as parameters in algorithms so that they work for a variety of suitable data types and data structures.

A significant benefit of object oriented programming is reusability of code which eliminates redundant coding. An important feature of C++ called templates strengthens this benefit of OOP and provides great flexibility to the language. Templates support generic programming, which allows to develop reusable software components such as functions, classes etc.. supporting different data types in a single framework. Instead of writing different functions for the different data types, we can define common function. For example

```
int max(int a,int b); // Returns maximum of two integers float
max(float a,float b); // Return maximum of two floats char
max(char a,char b); // Returns maximum of two characters
(This is called function overloading)
```

But, instead of writing three different functions as above, C++ provided the facility called "Templates". With the help of templates you can define only one common function as follows:

```
T max(T a,T b); // T is called generic data type
```

Template functions are the way of making function/class abstracts by creating the behavior of function without knowing what data will be handled by a function. In a sense this is what is known as “generic functions or programming”.

Template function is more focused on the algorithmic thought rather than a specific means of single data type.

For example you could make a templated stack push function. This push function can handle the insertion operation to a stack on any data type rather than having to create a stack push function for each different type.

Syntax:

```
template < class type >
```

```
ret_type fun_name(parameter list)
{
- -----//body of function
}
```

Features of templates:-

1. It eliminates redundant code
2. It enhances the reusability of the code.
3. It provides great flexibility to language

Templates are classified into two types.

- 1.Function templates
- 2.Class Templates

Function Templates

The templates declared for functions are called as function templates. A function template defines how an individual function can be constructed.

Syntax :

```
template < class type,.....>
ret_type fun_name(arguments)
{
- -----// body of the function
}
```

Class Templates

The templates declared for classes are called class templates. A class template specifies how individual classes can be constructed similar to the normal class specification. These classes model a generic class which supports similar operations for different data types.

General Form of a Class Template:

```
template <class T>
class class-name
{
.....
.....
};
```

A class created from a class template is called a template class. The syntax for defining an object of a template class is:

```
classname<type> objectname(arglist);
```

CLASS TEMPLATE

```
#include<iostream.h>
#include<conio.h>
template <class T>
class swap
{
    T a,b;
public:
    swap(T x,T y)
    {
        a=x;
        b=y;
    }
void swapab()
{
    T temp;
    temp=a;
    a=b;
    b=temp;
}
void showdata()
{
    cout<<a<<b;
}
};
void main()
{
    int m,n;
    float m1,n1;
    cout<<"Enter integer values";
    cin>>m>>n;
    cout<<"Enter floating values";
    cin>>m1>>n1;
    swap<int> c1(m,n);
    swap<float> c2(m1,n1);
    c1.swapab();
    c1.showdata();
    c2.swapab();
    c2.showdata();
}
```

Class Template with Multiple Parameters

Syntax:

```
template <class T1, class T2,...>
class class-name
{
.....
.....
}
#include<iostream.h>
```

```
template <class T1,class T2>
class Test
{
T1 a;
T2 b;
public:
Test(T1 x,T2 y)
{
a=x;
b=y;
}
void show()
{
cout<<a<<b;
}
};
void main()
{
Test<float,int> test1(1.23,123);
Test<int,char> test2(100,"w");
test1.show();
test2.show();
}
```

FUNCTION TEMPLATES

Like class template we can also define function templates that would be used to create a family of functions with different argument types.

General Form:

```
template <class T>
return-type function-name (arguments of type T)
{
```

.....

.....

```
}
```

```
#include<iostream.h>
```

```
template<class T>
```

```
void swap(T &x, T &y)
```

```
{
```

```
T temp = x;
```

```
x=y;
```

```
y=temp;
```

```
}
```

```
void fun(int m,int n,float a,float b)
```

```
{
```

```
cout<<m<<n;
```

```
swap(m,n);
```

```
cout<<m<<n;
```

```
cout<<a<<b;
```

```
swap(a,b);
```

```
cout<<a<<b;
```

```
}
```

```
int main()
```

```
{
```

```
fun(100,200,11.22,33.44);
```

```
return 0;
```

```
}
```

Example:-

```
#include < iostream.h >
```

```
#include < conio.h >
```

```
template
```

```
T max(T a, T b)
```

```
{
```

```
if(a>b)
```

```

return a;
else
return b;
}
void main( )
{
char ch1,ch2,ch3;
cout<<"enter two characters"<< ch2<< ch3;
cin>>ch2>>ch3;
d=max(ch2,ch3);
cout<<"max(ch2,ch3)"<< ch1;
int a,b,c;
cout<<"enter two integers:";
cin>>a>>b;
c=max(a,b);
cout<<"max(a,b):"<< c<< endl;
float f1,f2,f3;
cout<<"enter two floats< f1 f2 >:";
cin>>f1,f2;
f3=max(f1,f2);
cout<<"max(f1,f2):"<< f3;
}

```

Output:

```

enter two characters: A,B
max(ch2,ch3):B
enter two integers:20,10
max (a,b) :20
enter two floats :20.5,30.9
max (f1,f2) :30.9

```

Function Template with Multiple Parameters

Like template class, we can use more than one generic data type in the template statement, using a comma-separated list as shown below:

```

template <class T1, class T2,> return-type function_name(arguments of
types T1,T2.)
{
.....
.....
}
#include<iostream.h>

```

```

#include<string.h>
template<class T1, class T2>
void display(T1 x,T2 y)
{
cout<<x<<y;
}
int main()
{
display(1999,"EBG");
display(12.34,1234);
return 0;
}

```

File Handling in C++

File handling means reading from and writing to files (like .txt, .csv, etc.) using classes provided by the C++ standard library.

- Programs run in RAM, meaning data exists only while the program is running, when a program ends, all data in RAM is lost automatically.
- File handling allows storing data in secondary memory (like HDD or SSD), so it can be preserved permanently and can be saved and accessed even after the program terminates.
- For file operations, C++ provides file stream classes in the <fstream> header such as ofstream, ifstream, fstream.

Opening a File

Before reading from or writing to a file, we first need to open it. Opening a file loads that file in the RAM. In C++, we open a file by creating a stream to it using the fstream class that represent the file stream i.e. stream for input and output to the file.

```

fstream str("filename.ext", mode);
where,

```

str: Name given to the stream

filename: Name of the file

mode: Represents the way in which we are going to interact with the file.

File Opening Modes

File opening mode indicate file is opened for reading, writing, or appending. Below is the list of all file modes in C++:

Mode	Description
<code>ios::in</code>	File open for reading. If file does not exists, the open operation fails.
<code>ios::out</code>	File open for writing: the internal stream buffer supports output operations.
<code>ios::binary</code>	Operations are performed in binary mode rather than text.
<code>ios::ate</code>	The output position starts at the end of the file.
<code>ios::app</code>	All output operations happen at the end of the file, appending to its existing contents.
<code>ios::trunc</code>	Any contents that existed in the file before it is open are discarded.

For Example, if we want to open the file for reading, we use the following opening mode:

```
fstream filein("file.txt", ios::in);
```

Similarly, if we want to open the file for writing, we use the following:

```
fstream fileout("file.txt", ios::out);
```

These modes can also be combined using OR operator (`|`). For example, you can open the file stream in both read and write mode as shown:

```
fstream str("file.txt", ios::in | ios::out);
```

If the file opened in write mode does not exists, a new file is created. But if the file opened in read mode doesn't exists, then no new file is created, and an exception is thrown

Other File Streams

fstream is not the only file stream provided by C++. There are two more specialized streams:

- ifstream: Stands for input file stream. It is equivalent to open fstream in ios::in mode.
- ofstream: Stands for output file stream. It is equivalent to opening fstream in ios::out mode.

The above modes are default modes for these streams. These modes cannot be changed but can be clubbed together with other modes. Now for input, we can also use ifstream as shown:

```
ifstream filein("file.txt");
```

Similarly, for output:

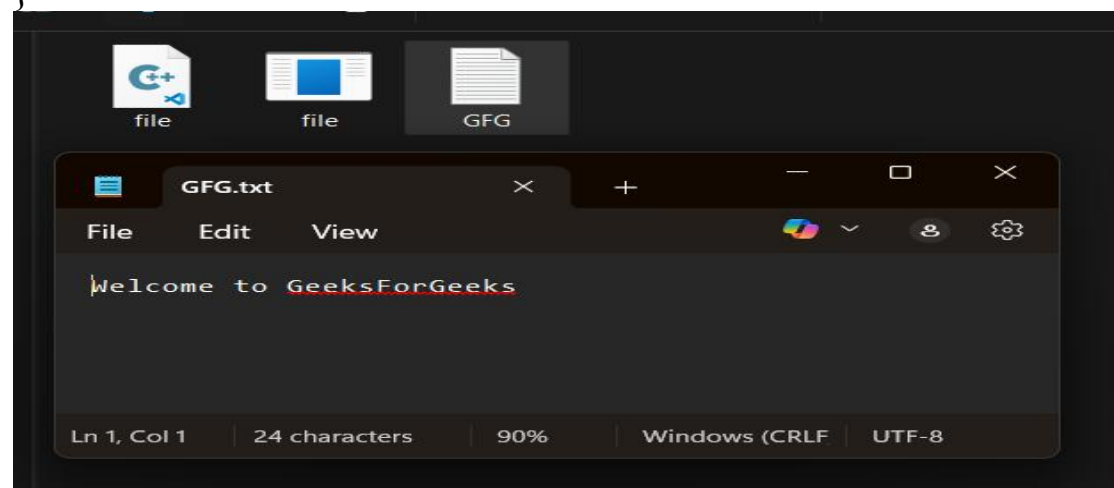
```
ofstream fileout("file.txt");
```

Write Data to File

Once the file is opened in the write mode using either fstream or ofstream, we can perform the write operation in similar way as with cout using << operator.

```
#include <bits/stdc++.h>
using namespace std;
```

```
int main() {
    // Open a file
    ofstream file("GFG.txt");
    // Write the string to the file
    file << "Welcome to GeeksforGeeks.";
    return 0;
}
```



Read Data from File

Once the file is opened in the read mode using either `fstream` or `ifstream`, we can perform the write operation in similar way as with `cin` using `>>` operator.

```
#include <bits/stdc++.h>
using namespace std;
int main()
{
    // Open a file in read mode
    ifstream file("GFG.txt");
    string s;

    // Read string from the file
    file >> s;
    cout << "Read String: " << s;
    return 0;
}
```

Output

Read String: Welcome

This has same problem as `cin`. The input is only taken until the first whitespace character. To avoid this, we can use the `getline()` function as shown:

```
#include <bits/stdc++.h>
using namespace std;
int main()
{
    // Open a file in read mode
    ifstream file("GFG.txt");
    string s;
    // Read string from the file
    getline(file, s);
    cout << "Read String: " << s;
    return 0;
}
```

Output

Read String: Welcome to GeeksforGeeks.

Closing the File

Closing the file means closing the associated stream and free the resources that we being used. It is important to close the file after you are done with it, especially in the long running programs to avoid memory leaks, data loss, etc.

In C++, the files are closed using the `close()` member function that is present in all file streams.

```
#include <bits/stdc++.h>
using namespace std;

int main()
{

    // Open a file in read mode
    ifstream file("GFG.txt");
    string s;

    // Read string from the file
    getline(file, s);

    cout << "Read String: " << s;

    // Close the file
    file.close();

    return 0;
}
```

Output

Read String: Welcome to GeeksforGeeks.

Errors in File Handling

Many different types of errors can occur in file handling such as file not found, disk full, etc. Our programs should expect common errors and should be able to handle them properly. Following are some common errors that can occur during file handling:

File Open Failure

There can be cases in which the file is not opened due to various reasons such as it doesn't exists, or the program does not have permission to open

it, etc. In this case, we can use the `is_open()` member function of the file stream classes to check whether the file is opened successfully or not.

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    fstream file("nonexistent_file.txt", ios::in);

    // Check if the file is opened
    if (!file.is_open()) {
        cerr << "Error: Unable to open file!" << endl;
        return 1;
    }
    file.close();
    return 0;
}
```

Output

Error: Unable to open file!

Failure to Read/Write Data

Another common error is failure to read or write data for reasons such as incorrect mode, etc. In this case, we can validate operations after each read/write attempt. For example, reading using `getline()` can be validated as shows:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    fstream file("GFG.txt", ios::out);

    if (!file.is_open()) {
        cerr << "Error: Unable to open file!" << endl;
        return 1;
    }
    string line;

    // Checking if getline() read successfully or not
    if (!getline(file, line))
        cerr << "Error: Failed to read data" << endl;
```

```
    file.close();  
    return 0;  
}
```

Output

Error: Failed to read data

End-of-File (EOF) Error

Trying to read beyond the end of the file can cause an EOF error. This can happen when you don't check for the end of the file before reading. We can check for EOF using eof() member function.

```
#include <bits/stdc++.h>  
using namespace std;  
int main()  
{  
    ifstream file("GFG.txt");  
  
    if (!file.is_open())  
    {  
        cerr << "Error: Unable to open file!" << endl;  
        return 1;  
    }  
    string line;  
    while (getline(file, line))  
        cout << line << endl;  
  
    // Check for eof  
    if (file.eof())  
        cout << "Reached end of file." << endl;  
    else  
        cerr << "Error: File reading failed!" << endl;  
  
    file.close();  
    return 0;  
}
```

Output

Reached end of file.

Notice that we have also validated the read operation before checking EOF as getline() will only return nullptr even if the read fails due to any reason.

Handling Binary Files

In C++, we can also handle binary files, which store data in raw format. To read and write binary data, must use the `ios::binary` flag when creating/opening a binary file.

Write into Binary File

To write data to a binary file, we first need to open or create the file in `ios::binary` mode.

```
#include <cstring>
#include <fstream>
#include <iostream>
using namespace std;
int main()
{
    string str = "Welcome to GeeksForGeeks";

    // Open a binary file for writing
    ofstream file("fileBin.bin", ios::binary);

    // Check if the file is open
    if (!file)
    {
        cerr << "Error opening the file for writing.";
        return 1;
    }

    // Write the length of the string (size) to file first
    size_t strLength = str.length();
    file.write(reinterpret_cast<const char *>(&strLength),
    sizeof(strLength));

    // Write the string to the binary file
    file.write(str.c_str(), strLength);

    // Close the file
    file.close();
    return 0;
}
```

Output

writeBinary
Binary File

Reading from Binary File

Just as we open a file in binary mode to write data, to read data from a binary file, we must open the file in read mode using `ios::in`.

Syntax:

```
fstream fileInstance("fileName.bin", ios::in | ios::binary);
```

```
#include <cstring>
#include <fstream>
#include <iostream>
using namespace std;
int main()
{
    string str;
    // Open the binary file for reading
    fstream file("fileBin.bin", ios::in | ios::binary);
    // Check if the file is open
    if (!file)
    {
        cerr << "Error opening the file for reading.";
        return 1;
    }

    // Read the length of the string (size) from the file
    size_t strLength;
    file.read(reinterpret_cast<char *>(&strLength), sizeof(strLength));

    // Allocate memory for the string and read the data
    char *buffer = new char[strLength + 1]; // +1 for the null-terminator
    file.read(buffer, strLength);
    // Null-terminate the string
    buffer[strLength] = '\0';
    // Convert the buffer to a string
    str = buffer;
    // Print file data
    cout << "File Data: " << str;
    delete[] buffer;
    file.close();
    return 0;
}
```

Output

File Data: Welcome to GeeksForGeeks

Exception handling

Exceptions: Exceptions are runtime anomalies or unusual conditions that a program may encounter while executing. Anomalies might include conditions such as division by zero, accessing an array outside of its bounds or running out of memory or disk space. When a program encounters an exception condition, it must be identified and handled.

Exceptions provide a way to transfer control from one part of a program to another. C++ exception handling is built upon three keywords: try, catch, and throw.

Types of exceptions:

1. Synchronous exceptions
2. Asynchronous exceptions

1. Synchronous exceptions: Errors such as “Out-of-range index” and “over flow” are synchronous exceptions.

2. Asynchronous exceptions: The errors that are generated by any event beyond the control of the program are called asynchronous exceptions.

The purpose of exception handling is to provide a means to detect and report an exceptional circumstance.

Exception Handling Mechanism:

An exception is said to be thrown at the place where some error or abnormal condition is detected. The throwing will cause the normal program flow to be aborted, in a raised exception. An exception is thrown programmatically, the programmer specifies the conditions of a throw.

In handled exceptions, execution of the program will resume at a designated block of code, called a catch block, which encloses the point of throwing in terms of program execution. The catch block can be, and usually is, located in a different function than the point of throwing.

C++ exception handling is built upon three keywords: try, catch, and throw.

Try is used to preface a block of statements which may generate exceptions. This block of statements is known as try block. When an exception is detected it is thrown by using throw statement in the try

block. Catch block catches the exception thrown by throw statement in the try block and handles it appropriately.

```
#include<iostream>
using namespace std;
int main()
{
    int a,b;
    cout<<"Enter any two integer values";
    cin>>a>>b;
    int x=a-b;
    try
    {
        if(x!=0)
        {
            cout<<"Result(a/x)="<<a/x<<endl;
        }
        else
        {
            throw x;
        }
    }
    catch(int ex)
    {
        cout<<"Exception caught:Divide By Zero \n";
    }
}
```

THROWING MECHANISM:

When an exception is detected, it can be thrown by using throw statement in any one of the following forms

- throw(exception);
- throw exception;
- throw;

CATCHING MECHANISM:

```
Catch(data type arg)
{
    //statements for handling
    //exceptions
}
```

Multiple catch statements:

```
try
{
//try block
}
catch(data type1 arg)
{
//catch block1
}
catch(data type2 arg)
{
//catch block2
}
.....
.....
catch(data typeN arg)
{
//catch blockN
}
```

- When an exception is thrown, the exception handler is searched in order for an appropriate match.
- It is possible that arguments of several catch statements match the type of an exception. In such cases the first handler that matches the exception type is executed.

Write a Program to catch multiple catch statements.

```
#include<iostream.h>
void test(int x)
{
try

{
if(x==1) throw x;
else
if(x==0) throw 'x';
else
if(x==-1) throw 1.0;
cout<<"end of try block"<<endl;
}
catch(char c)
{
cout<<"caught a character"<<endl;
```

```

    }
    catch(int m)
    {
        cout<<"caught an integer"<<endl;
    }
    catch(double d)
    {
        cout<<"caught a double"<<endl;
    }
}
int main()
{
    test(1);
    test(0);
    test(-1);
    test(2);
    return 0;
}

```

Output:

```

caught an integer
caught a character
caught a double
end of try block

```

Catch All Exceptions:

It may not be able to design independent catch handlers to catch them. In such circumstances, we can force a catch statement to catch all exceptions instead of a certain type alone.

```

catch(...)
{
    .....
}

```

Write a Program to catch all exceptions.

```

#include<iostream.h>
void test(int x)
{
    try
    {
        if(x==0) throw x;
        if(x==0) throw 'x';
        if(x== -1) throw 1.0;
    }
}

```

```

    }
    catch(...)
    {
        cout<<"caught exception"<<endl;
    }
}
int main()
{
    test(-1);
    test(0);
    test(1);
    return 0;
}

```

Re-throwing an Exception:

It is possible to pass exception caught by a catch block again to another exception handler. This is known as Re-throwing.

```

#include <iostream>
using namespace std;
void MyHandler()
{
    try
    {
        throw "hello";
    }
    catch (const char*)
    {
        cout <<"Caught exception inside MyHandler\n";
        throw; //rethrow char* out of function
    }
}
int main()
{
    cout<< "Main start... "<<endl;
    try
    {
        MyHandler();
    }
    catch(const char*)
    {
        cout <<"Caught exception inside Main\n";
    }
    cout << "Main end";
}

```

```

    return 0;
}

```

Specifying Exceptions:

Specification of exception restrict functions to throw some specified exceptions only with the use of throw(exception list) in the header of the function.

General form

Type function_name(argument list) throw(exception -list)

```

{
Statements

```

```

try
{
    statements
}
}

```

```

#include <iostream>
using namespace std;
void test(int x) throw(int,float,char)
{
switch(x)
{
case 1:throw x;
break;
case 2:throw 'x';
break;
case 3:throw double(x);
break;
case 4:throw float(x);
break;
}
}
int main()
{
try
{
test(4); //test(4) leads to abnormal termination
}
catch(int i)
{
cout << "Caught int type exception\n";
}
}

```

```
    }  
catch(float f)  
{  
    cout <<"Caught float type exception\n";  
}  
catch(char c)  
{  
    cout <<"Caught char type exception\n";  
}  
catch(double i)  
{  
    cout <<"Caught Double type exception\n";  
}  
return 0;  
}
```